

Model Comparison and Performance Analysis

The experimental results showed that **lightweight and balanced architectures** performed better on the available dataset than highly complex models.

Since the dataset size was relatively small (around **600 images**), models with lower complexity achieved better generalization and more stable learning.



CNN + TF-IDF + ANN (Baseline Model)



Advantages

- Fast training
- Simple architecture
- Low computational cost
- Good baseline for comparison

Disadvantages

- TF-IDF does not understand contextual meaning
- Limited text representation
- Basic image feature extraction

i Performance: The model achieved good accuracy because its simplicity reduced the risk of overfitting on the small dataset.

MobileNetV2 + GRU

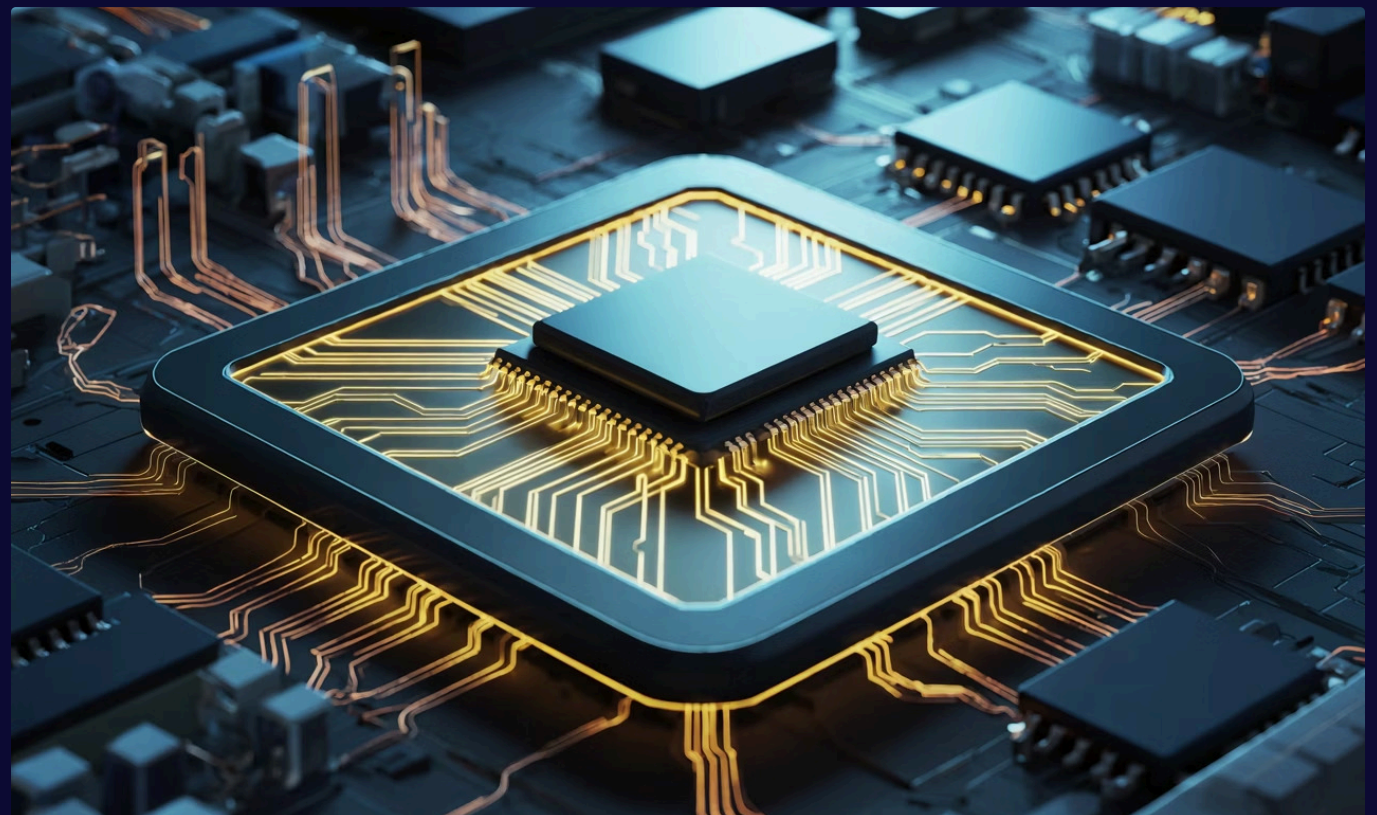
Advantages

- Lightweight architecture
- Fast and efficient training
- Lower memory usage
- Good balance between speed and accuracy
- Suitable for small datasets

Disadvantages

- GRU captures context less effectively than LSTM
- Lower representation capability than deeper models

- ✓ **Performance:** This model achieved very strong performance because MobileNetV2 extracted efficient image features while keeping the architecture lightweight and stable during training.



MobileNetV2 + LSTM

Advantages

- Better contextual understanding
- Improved text sequence learning
- Efficient image feature extraction
- Balanced complexity

Disadvantages

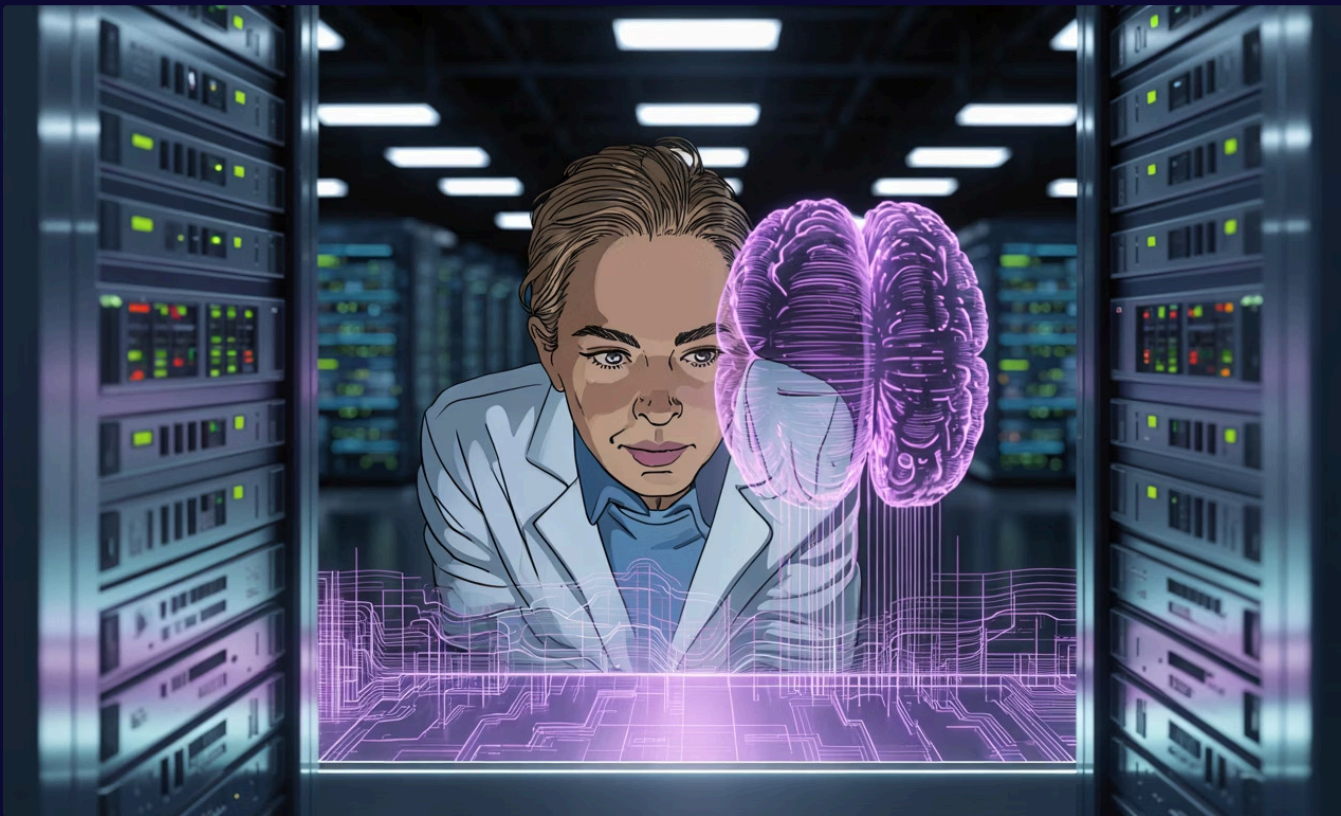
- Slightly slower than GRU
- Higher computational cost

Performance

This model achieved the **best overall performance** among all experiments. The combination of MobileNetV2 and LSTM created an effective balance between feature extraction, contextual understanding, and model complexity.

The architecture was powerful enough to learn useful multimodal patterns while still being suitable for the limited dataset size.

ResNet50 + LSTM



Advantages

- Strong image feature extraction
- Deep visual understanding
- Powerful multimodal representation

Disadvantages

- Large architecture
- Longer training time
- Higher memory consumption
- Increased overfitting risk

⚠ **Performance:** Although the model achieved high accuracy, its larger complexity made training more difficult on the small dataset compared to MobileNetV2-based models.

ResNet50 + BERT

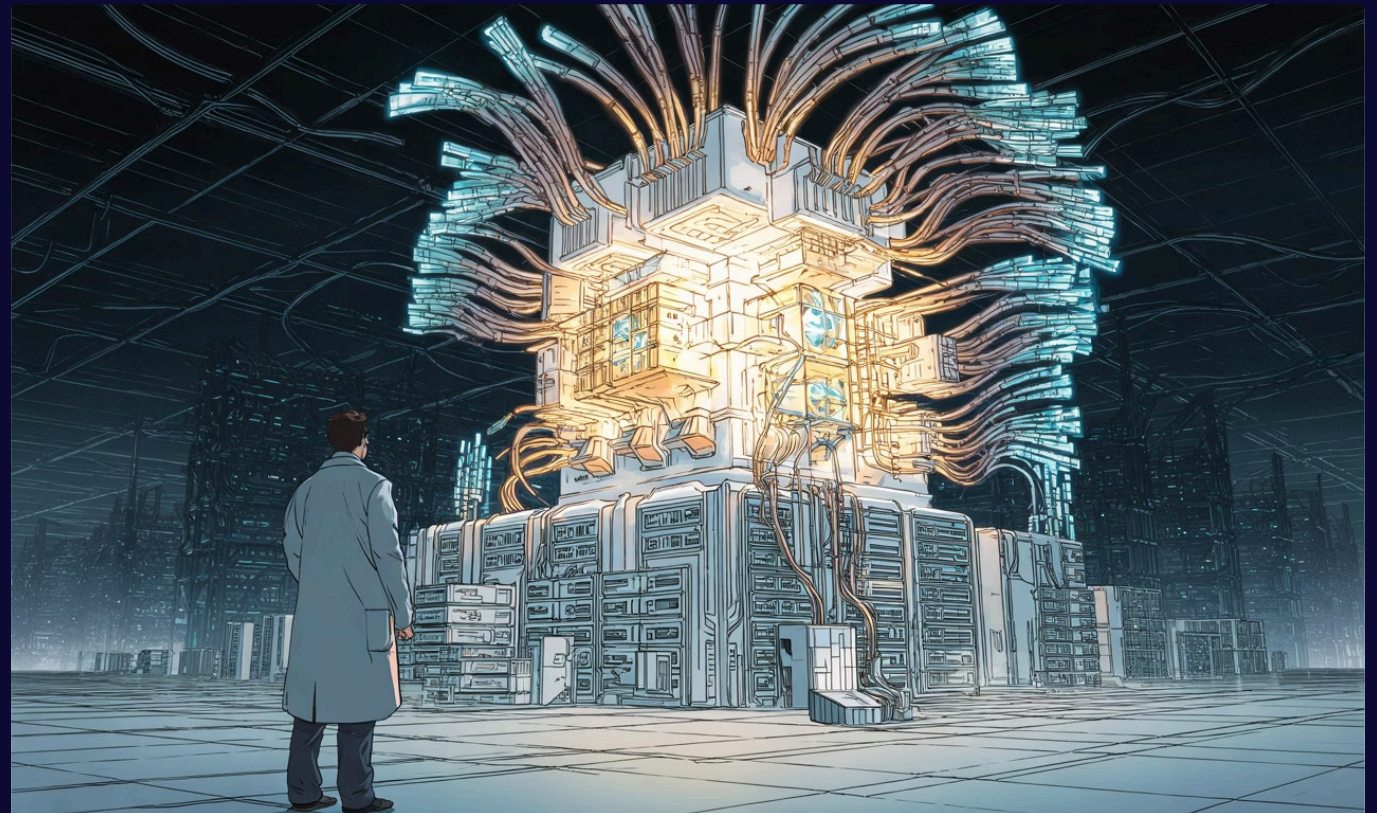
Advantages

- Advanced contextual understanding
- Powerful semantic representation
- State-of-the-art architecture

Disadvantages

- Very high computational complexity
- Requires larger datasets
- More sensitive to overfitting
- Slower training process

⊗ **Performance:** This model achieved lower accuracy than expected because the dataset size was not sufficient for effective fine-tuning of large architectures such as BERT and ResNet50.



Final Comparison

A side-by-side view of all five models across key evaluation dimensions:

Model	Complexity	Speed	Generalization	Accuracy
CNN + TF-IDF + ANN	Low	Very Fast	Good	Good
MobileNetV2 + GRU	Medium	Fast	Very Good	High
MobileNetV2 + LSTM	Medium	Fast	Excellent	Highest
ResNet50 + LSTM	High	Slow	Good	High
ResNet50 + BERT	Very High	Very Slow	Lower	Lower

Final Conclusion

The experiments demonstrated that **model performance does not only depend on using deeper or more complex architectures.**

The best-performing model was **MobileNetV2 + LSTM** because it achieved the best balance between:



Accuracy



Computational Efficiency



Contextual Understanding



Generalization Ability

This architecture was more suitable for the relatively small multimodal dataset compared to larger models such as ResNet50 + BERT.

Supporting Evidence

Key screenshots and results from the experimental runs:

```
arn.metrics import f1_score
del.predict([X_img_test, X_test])
ed.argmax(axis=1)
core(y_test, pred, average='weig
Score:", f1)
```

1s 219ms/step
0.7320150862068966

6s 2s/step

precision	recall	f1-score	support
0.92	0.86	0.89	14
0.73	0.80	0.76	10
0.86	0.86	0.86	14
1.00	0.92	0.96	12
1.00	1.00	1.00	12
0.89	0.94	0.92	18
0.90	0.90	0.90	80
0.90	0.90	0.90	80
0.90	0.90	0.90	80

2]
0]
0]
0]
0]
0]
17]]

```
del.evaluate(
accuracy)
/step - accuracy: 0.7625 - loss: 0.5286
```

Report:

precision	recall	f1-score	support
1.00	0.86	0.92	14
0.83	1.00	0.91	10
1.00	0.57	0.73	14
1.00	1.00	1.00	12
0.67	1.00	0.80	12
0.89	0.89	0.89	18
		0.88	80
0.90	0.89	0.87	80
0.90	0.88	0.87	80

cessfully 

on_report(all_labels, all_preds))

precision	recall	f1-score	support
0.80	0.94	0.86	17
1.00	0.46	0.63	13
0.92	0.86	0.89	14
1.00	1.00	1.00	8
1.00	0.36	0.53	14
0.43	0.86	0.57	14
		0.74	80
0.86	0.75	0.75	80
0.84	0.74	0.73	80